

Assignment 3 - Concurrency, Shared Memory, Virtual Memory, Files

SYSC4001 - L1

Kalyah McKesey - 101307188

Sarah Andrew - 101303468

Part 1

[sarahandrew1012/SYSC4001_A3_P1](https://github.com/sarahandrew1012/SYSC4001_A3_P1)

Purpose

This simulator implements a system scheduler that manages processes using three algorithms. The algorithms implemented were round robin, external priority, and a combination of external priority and round robin scheduler. The round-robin scheduler allocates an equal time slice to each process. When the process doesn't finish within its time slice, it is preempted by another ready process. External priority operates by selecting the highest-priority task; thus, the smallest PID executes first and runs until completion. The combined external priority and round-robin algorithm selects the highest-priority process to run. When the process reaches the quantum, the round-robin preempts it.

Each process contains a trace file which includes the pid, memory size, arrival time, total CPU Time, I/O Frequency, and I/O Duration. These inputs were used to simulate different scenarios, including mostly CPU-bound, I/O-bound, and a combination of both. After executing the trace, an execution file was generated that outlines the process-state transition behaviour. These transitions include those from new, ready, running, waiting, and terminated states—essentially reflecting the behaviour of the schedulers implemented.

Discussion of Test Results

Simulation 1: Mostly CPU-Bound (Trace file 8)

External Priority:

Execution File:

In the execution file, the external process scheduler begins with PID 10, which arrives at time 0. The process then transitions from new -> ready -> running as it is the only process selected, until it reaches time 50. At time 50, a process with the PID of 1 begins the process-state transition. In external priority, it operates on a non-preemptive basis; thus, PID 10 finishes at time 250. After PID 10 completes, the external priority will select a new process and run it until termination.

In a CPU-bound scenario with external priority, the scheduler runs the already-running process, as external priority is non-preemptive scheduler. This causes the first process, PID 10, with the longest burst time, to execute to completion. When there is no I/O and a strictly CPU-bound workload, the priority begins to take effect. Thus, external priority works best when the process

with the smallest PID arrives first, since preemption does not occur for the process that begins execution first.

Average Throughput (Number of processes / Total Time): $\frac{2}{400}$

Average Turnaround Time (Completion - Arrival): $\frac{250+350}{2} = 300 \text{ ms}$

Average Waiting Time (Turnaround - Burst Time): $\frac{0+200}{2} = 100 \text{ ms}$

Average Response time: $\frac{0+200}{2} = 100 \text{ ms}$

RR:

Execution File:

In the execution file, it begins with PID 10, which executes within the quantum of 100 ms. The process continues until time 50, as it is the only process in the ready queue. From 0-50, PID 10 is the only process; therefore, it receives the quantum in slices. At time 50, PID 1 arrives. This process executes from 250 to 350, then resumes after 350.

In a CPU-bound scenario with RR, the scheduler behaves the same as FCFS when there is only one process in the ready queue. It performs the execution in consecutive 100 time slices without preemption. Therefore, it produces the same results as external priority.

Average Throughput (Number of processes / Total Time): $\frac{2}{400}$

Average Turnaround Time(Completion - Arrival): $\frac{250+350}{2} = 300 \text{ ms}$

Average Waiting Time(Turnaround - Burst Time): $\frac{0+200}{2} = 100 \text{ ms}$

Average Response time: $\frac{0+200}{2} = 100 \text{ ms}$

RR + EP:

Execution File:

In the execution file, the combined external priority and round-robin scheduler starts with PID 10. It performs the following transitions from new to ready, then to running, and executes until time 50. At time 50, process 1 arrives and is inserted into the ready queue. Process 1 does not preempt PID 10 during the burst execution.

In a CPU-bound scenario with EP + RR, it selects the process with the highest priority. However, the current process does not get preemptive until the quantum expires. Since PID 10 has a longer CPU burst than PID 1, it dominates the CPU. As a result, it produces the same functionality as EP and RR.

In terms of turnaround time, waiting time, and response time, across all schedulers yielded the same results. This is consistent with a process that is mainly CPU-bound and has no I/O events. When the second process arrives, the algorithm behaves as FCFS. This is because EP is a non-preemptive scheduler, and round-robin scheduling begins alternating when there is more than one process in the ready queue. As for the combination scheduler of external priority and round robin, the scheduler will use round robin after the priority section. In this case, round robin never triggers preemptive. In conclusion, when the workload is mostly CPU-bound and the second process starts after the first, the schedulers EP, RR, and EP + RR all produce the same results.

Average Throughput (Number of processes / Total Time): $\frac{2}{400}$

Average Turnaround Time (Completion - Arrival): $\frac{250+350}{2} = 300 \text{ ms}$

Average Waiting Time(Turnaround - Burst Time): $\frac{0+200}{2} = 100 \text{ ms}$

Average Response time: $\frac{0+200}{2} = 100 \text{ ms}$

Simulation 2: Mostly I/O bounded (Trace 15)

EP:

Execution file:

When analysing trace 15 with the external priority, the scheduler selects the smallest PID, so PID 3 executes first. As PID 3 has no I/O operations, it occupies the CPU and runs until it completes its 300-second burst. After PID 3 is terminated, the scheduler selects PID 5. PID 5 and PID 7 perform I/O operations every 30 ms, causing them to alternate between running, waiting, and ready. This behaviour reflects the external priority functionality as it is non-preemptive. Therefore, PID 5 and 7 alternate based on which I/O is completed first and becomes ready first.

In a scenario where I/O is mostly bounded, external priority selects the process with the smallest PID. This causes PID 3 to run until completion. In return, the I/O-Intensive processes, PIDs 5 and 7, yield higher waiting times. Therefore, external priority becomes less effective when there are many I/O activities, because I/O interruptions determine which process gets the CPU.

Average Throughput (Number of processes / Total Time): $\frac{3}{900}$

Average Turnaround Time(Completion - Arrival): $\frac{300+870+900}{3} = 690\text{ms}$

Average Waiting Time(Turnaround - Burst Time): $\frac{0+570+600}{3} = 390 \text{ ms}$

Average Response time: $\frac{0+300+330}{3} = 210 \text{ ms}$

RR:

Execution File:

Initially, when all three processes are placed in the ready queue with a quantum of 100 units, the processes are divided into equal time slices. At time 0, PID 7 is selected first, begins execution, and runs for 30 ms. Once PID 7 performs an I/O operation, the scheduler selects PID 5 as the next ready process. Lastly, since PID 3 has no I/O, it begins once PID enters the waiting state. Thus, the scheduler focuses on the process that becomes ready first after I/O completes. The system repeats the cycle until an all-time slice has been used.

When processes are mostly I/O bounded with RR, the round robin scheduler performs efficiently. As PID 5 and 7 leave the CPU when completing I/O, it gives a fair amount of execution time to all processes. This demonstrates how round-robin fairness works and how I/O-intensive processes execute without delay. However, CPU-bound processes experience longer wait and response times.

Average Throughput (Number of processes / Total Time): $\frac{3}{918}$

Average Turnaround Time(Completion - Arrival): $\frac{588 + 618 + 918}{3} = 708 \text{ ms}$

Average Waiting Time(Turnaround - Burst Time): $\frac{288 + 318 + 618}{3} = 408 \text{ ms}$

Average Response time: $\frac{0 + 31 + 618}{3} = 216.3 \text{ ms}$

EP + RR:

The external priority and round-robin scheduler determines which scheduler to execute based on the priority, then applies round-robin among the processes with that priority. This causes PID 3 to perform first for 100 units, as it has the highest priority. PID 3 gets preempted every 100 ms. Once PID 3 reaches 300 ms of execution, PID 5 and 7 begin their execution. When PID 5 and PID 7 are executing, they exhibit the same behaviour as the external priority in which PID 5 and 7 alternate based on I/O completion.

When processes are mostly I/O-bounded and use external priority and round-robin scheduling, the process with the highest priority begins first. From there, the CPU bursts are preemptive. Thus, external priority and round robin behave similarly to external priority once the first process completes.

Average Throughput (Number of processes / Total Time): $\frac{3}{900}$

Average Turnaround Time(Completion - Arrival): $\frac{300 + 870 + 900}{3} = 690 \text{ ms}$

Average Waiting Time(Turnaround - Burst Time): $\frac{0 + 570 + 600}{3} = 390 \text{ ms}$

Average Response time: $\frac{0 + 300 + 330}{3} = 210 \text{ ms}$

In terms of the metrics computed, round robin has the lowest response time, a higher throughput value, as the CPU remains busy during process switches, and the lowest average response time, as PID 5 and 7 resume after an I/O completion. External priority is less effective, as it results in higher I/O wait times for I/O-bound processes and lower response times. This is because PID 3 runs until its burst is completed. External + round-robin scheduling performs better than external priority in I/O-bound cases; however, it is not as responsive as round-robin scheduling. In conclusion, processes that are mostly I/O-bounded benefit most from round-robin scheduling, whilst processes with external priorities perform poorly.